1. g(n): Depth of the node, h(n): Number of Misplaced tiles

```
import heapq

# Goal State representation (0 represents the empty tile)
GOAL_STATE = (
    1, 2, 3,
    4, 5, 6,
    7, 8, 0
)

# Allowed movements for blank tile: (row_delta, col_delta, Action_Name)
MOVES = [
    (-1, 0, 'Up'),
    (1, 0, 'Down'),
    (0, -1, 'Left'),
    (0, 1, 'Right')
]

def print_board(state):
    """Utility function to display the board."""
    for i in range(0, 9, 3):
        print(f" {state[i]} {state[i+1]} {state[i+2]} ")
    print()

def heuristic_misplaced_tiles(state, goal=GOAL_STATE):
    """
    h(n): Count of tiles that are not in their target goal position.
```

```
    The empty tile (0) is ignored by standard convention.
    """
    misplaced = 0
    for i in range(9):
        if state[i] != 0 and state[i] != goal[i]:
            misplaced += 1
    return misplaced

def get_neighbors(state):
    """Generates all valid successor states from current state."""
    neighbors = []
    zero_idx = state.index(0)
    row, col = divmod(zero_idx, 3)

    for dr, dc, move_name in MOVES:
        new_row, new_col = row + dr, col + dc
        if 0 <= new_row < 3 and 0 <= new_col < 3:
            new_idx = new_row * 3 + new_col
            state_list = list(state)
            # Swap blank tile with the adjacent tile
            state_list[zero_idx], state_list[new_idx] = state_list[new_idx], state_list[zero_idx]
            neighbors.append((tuple(state_list), move_name))

    return neighbors

def a_star_misplaced(start_state, goal_state=GOAL_STATE):
    """
    Solves the 8-puzzle using A* search with h(n) = misplaced tiles.
```

```
Priority Queue stores: (f, g, state, path)
"""
# Priority Queue (Min-Heap)
h_start = heuristic_misplaced_tiles(start_state, goal_state)
frontier = []
heapq.heappush(frontier, (h_start, 0, start_state, []))

# Keep track of lowest g(n) found for visited states
cost_so_far = {start_state: 0}
nodes_expanded = 0

while frontier:
    f, g, current_state, path = heapq.heappop(frontier)
    nodes_expanded += 1

    # Goal check
    if current_state == goal_state:
        return path, g, nodes_expanded

    # If a cheaper path to this state was already processed, skip
    if g > cost_so_far.get(current_state, float('inf')):
        continue

    for neighbor, move in get_neighbors(current_state):
        new_g = g + 1  # Depth increases by 1 for each move

        if neighbor not in cost_so_far or new_g < cost_so_far[neighbor]:
            cost_so_far[neighbor] = new_g
```

```
            h = heuristic_misplaced_tiles(neighbor, goal_state)
            new_f = new_g + h
            heapq.heappush(frontier, (new_f, new_g, neighbor, path + [move]))

    return None, None, nodes_expanded


# ========================================
# Example Execution
# ========================================
if __name__ == "__main__":
    initial_state = (
        1, 2, 3,
        4, 0, 6,
        7, 5, 8
    )

    print("--- Initial State ---")
    print_board(initial_state)

    print("--- Goal State ---")
    print_board(GOAL_STATE)

    initial_h = heuristic_misplaced_tiles(initial_state)
    print(f"Initial Misplaced Tiles h(start): {initial_h}\n")

    path, total_cost, expansions = a_star_misplaced(initial_state, GOAL_STATE)
```

```
if path is not None:
    print(f"Goal Reached!")
    print(f"Optimal Depth / Cost g(n): {total_cost}")
    print(f"Total Nodes Expanded: {expansions}")
    print(f"Move Sequence: {' -> '.join(path)}")
else:
    print("No solution found.")
```

```
--- Initial State ---
 1 2 3
 4 0 6
 7 5 8

--- Goal State ---
 1 2 3
 4 5 6
 7 8 0

Initial Misplaced Tiles h(start): 2

Goal Reached!
Optimal Depth / Cost g(n): 2
Total Nodes Expanded: 3
Move Sequence: Down -> Right
```

[ ]: